

Section 1: Comprehensive Case Scenario Assignments (Basic to Advanced)

Case Scenario 1: E-Commerce Dynamic Shipping Engine

- **Core Concept:** Control Structures (if-elif-else and nested conditions)
- **Difficulty:** Basic
- **Scenario:** An online store calculates delivery charges dynamically. Orders with a cart value over \$150 receive free shipping. If the cart value is under \$150, look up the destination region: "Domestic" shipping costs a flat fee of \$10, while "International" shipping costs \$25. However, if the customer holds a premium membership pass, they receive a \$5 discount on any calculated shipping fee under \$150, but the final shipping cost can never drop below \$0.
- **Task:** Code a program that asks the user to input their cart total, geographic delivery zone, and premium member status, then calculates and prints the precise final shipping cost.

Case Scenario 2: Smart Inventory Auditing and Alert Matrix

- **Core Concept:** Data Structures (Lists, Dictionaries, and Mutability)
- **Difficulty:** Basic
- **Scenario:** A small electronics warehouse tracks its current stock items inside a Python dictionary, where keys are product IDs and values are dictionaries containing both item name and stock quantities (e.g., {'PROD101': {'name': 'Wireless Mouse', 'stock': 12}}).
- **Task:** Write a script that checks this tracking dictionary. If an item's stock count drops below 15 units, append its name to a list called `low_stock_warnings`. If an item has exactly 0 units left, move that product ID out of the inventory dictionary entirely and place it into a separate list named `out_of_stock_discontinued`.

Case Scenario 3: Automated Password Compliance Validator

- **Core Concept:** Loops (while loops with conditional exit control)
- **Difficulty:** Intermediate
- **Scenario:** A secure network gateway forces users to choose an alphanumeric passcode that fulfills two rigid compliance filters: it must be at least 8 characters long, and it cannot contain blank space characters.
- **Task:** Write a program that uses an indefinite loop to prompt the user to input a password. If the input fails to meet either constraint, print the exact reason for rejection and decrement a total variable called `allowed_attempts` (starting at 5). The loop should break immediately if a successful passcode is provided, or if total allowed attempts reach 0, print a security lockout warning.

Case Scenario 4: Smart Parking Meter Transaction Tracker

- **Core Concept:** Loops (for loops and list parsing)
- **Difficulty:** Intermediate
- **Scenario:** A commercial parking garage monitors timestamps via a list of daily vehicle durations in minutes: [45, 120, 310, 15, 640, 180]. The pricing rules dictate that any duration under 60 minutes is charged at \$5. Any duration between 60 and 240 minutes is billed at \$12. Anything exceeding 240 minutes is flag-rated as an all-day fee of \$30.
- **Task:** Loop through the data to parse these numbers into two distinct tracking categories: a dictionary summarizing how many vehicles fell into each price tier, and a running revenue total calculating how much cash the meter generated over the entire list duration.

Case Scenario 5: Modular Payroll Compensation Engine

- **Core Concept:** Functions (Positional arguments, Default values, and Return scopes)
- **Difficulty:** Intermediate
- **Scenario:** A corporate payroll processor requires a modular calculation module. The standard working schedule is a flat 40 hours per week. If an employee exceeds 40 hours, the extra hours must be evaluated at an overtime modifier rate.
- **Task:** Construct a function called `calculate_weekly_stipend` that accepts parameters for `base_hourly_pay`, `hours_worked`, and a default argument for `overtime_multiplier=1.5`. The function must cleanly return the total gross pay earned, keeping standard hours and overtime payouts separated internally before combining them.

Section 2: "Company IT Ticket" Helpdesk Simulator

Context for Students: You are a Junior IT Support Specialist at a growing tech company. The automated systems are throwing errors, and your team has captured the issues as helpdesk tickets. Your job is to write clean, crash-proof Python code to resolve them.

Ticket ID: #TCK-101 – Live Event Registration Crash

- **Severity:** High
- **Core Concepts:** try-except, ZeroDivisionError, if-else
- **Ticket Description:** "Our company event dashboard crashes with a ZeroDivisionError when a new workshop starts. The system tries to calculate the average score of feedback ratings. If a workshop has just opened and received 0 ratings, the code tries to divide the total points by 0 attendees, completely freezing the tracker script."
- **Task:** Write a function named `calculate_average_score(total_points, total_ratings)`. Inside, use a try-except block to catch the ZeroDivisionError. If the error occurs (because `total_ratings` is 0), the function must return 0.0. Otherwise, it should return the normal division result.

Ticket ID: #TCK-202 – Corrupted Server Log Scanner

- **Severity:** Medium
- **Core Concepts:** for loops, Type Conversion, try-except ValueError, continue
- **Ticket Description:** "An automated script reads server processing response times (in milliseconds) from a mixed data list. However, if a background lag spikes or bad data drops an empty string or a text phrase into the sequence, the script tries to convert it to an integer, crashes with a ValueError, and fails to process the rest of the day's logs."
- **Task:** You are given the broken log stream: `raw_logs = [45, "TIMEOUT", 120, "", 88, "ERROR_99"]`. Write a for loop to iterate through the list. Use a try-except block to attempt to convert each log entry to an integer (int). If it converts successfully, print "Processing log: X ms". If it throws a ValueError, print "Skipping corrupted entry", and use the continue keyword to ensure the loop keeps running for the remaining items.

Ticket ID: #TCK-303 – Employee Badge Scan Key Error

- **Severity:** High
- **Core Concepts:** Dictionaries, Membership Checking (in), Functions
- **Ticket Description:** "When employees scan their security badges at the office entrance, a Python script queries our active database dictionary. If a contractor with a temporary badge id or a mistyped ID scans at the turnstile, the script throws a fatal KeyError and freezes the entire security gate system mid-scan."
- **Task:** Write a function named `verify_badge(badge_dict, badge_id)`. The function must look at a database dictionary like `employee_db = {"ID101": "Shyam", "ID102": "Sita"}`. Use an if-else membership check (`if badge_id in badge_dict:`) to safely verify if the ID exists. If it matches, return the employee's name. If it is missing, return "Access Denied: Invalid Badge ID" instead of allowing the application to crash.

Ticket ID: #TCK-404 – Missing System Configuration Log

- **Severity:** Medium
- **Core Concepts:** File Handling ('r'), try-except, FileNotFoundError
- **Ticket Description:** "Our automated nighttime backup tool attempts to read a flat configuration file path called system_config.txt. If a systems engineer deletes or renames that configuration file during maintenance, our script throws an unhandled FileNotFoundError and completely stops the entire company backup pipeline."
- **Task:** Write a file-reading script using a with open() block that targets system_config.txt in read mode ('r'). Wrap this operation inside a localized try-except block. If the file is missing, catch the FileNotFoundError smoothly and print a safe tracking message: "Warning: Configuration file went missing. Utilizing fallback defaults." so the rest of the script can complete.